

Fast and Safe Strings Programmers Guide

For Safer and Efficient “C” Programming Development

Version 4.0

Programmers Guide

Fast and Safe “C” Strings *“Programmers Guide”*.

Second Edition (July, 1999)

Third Edition – First Release (February, 2007)

Fourth Edition – (August, 2026)

This is a major revision of, and obseletes, the
First Edition of the Fast and Safe “C” String
“Programmers Guide” February 2007

Copyright © Clement Clarke. 1989-2026.

All Rights Reserved. The contents of this publication may not be reproduced in any form in any means in part or in whole without the prior written consent of Clarke Computer Software.

Postal Address:

**8/16 Jamorneau Street,
Warracknabeal,
Victoria,
Australia 3933.**

Telephone	Mobile (+61) 0401 054 155
Email	clemclarke@gmail.com

Preface

This is the fourth edition of the Fast Safe C Strings. A major addition is the ability to use Record I/O to programs which enables much faster I/O to be performed. Record I/O is totally compatible with IBM's Variable Blocked mainframe system. The files can be simply copied to the mainframe, and used directly as RECFM=VB files.

In addition to the addition of Record I/O, the routines have been greatly simplified using new compiler facilities particularly the use of Static Inline code allows for the simplification of the original 1983 code. The PL/I Fixed Length string has been removed.

Portability of languages, operating systems, system utilities and applications programs is a highly desirable goal. While coding in Assembler is a must for maximum efficiency, high level languages definitely give the desired portability, *as long as you have an efficient and effective compiler and good string handling facilities.*

"C" is often used to provide this portability between machines, but it lacks the ability to use many of the string instructions provided by IBM 370 and Intel 808xx chips effectively - thus leaving *millions* of machines running up to 25 (repeat *twenty-five*) times slower than they ought to be running for string copies and compares. And string handling is required for most applications.

This version of these routines provided a generic set of macros that work on any ANSI compatible C compiler. These give speed improvements of about 5 fold over the usual:

```
while (*dest++=*src++);
```

The routines automatically check that the receiving string is long enough to hold that data and truncate if necessary.

The routines in this package:

- Speed copying strings by a factor of up to 25 times through the use Assembler Macros. Generic "C" macros speed Copying about 5 times.
- Add a generic **CPY** function for copying one string to another.
- Automatically truncate long strings when they are copied to short strings so that storage following the shorter strings is not accidentally overwritten.
- Add a generic **CMP** function for comparing strings with each other, and optionally checking that the longer strings have blanks on the end thus providing a true string comparison.

CONTENTS

FAST AND SAFE STRINGS PROGRAMMERS GUIDE	1
PREFACE	3
INTRODUCTION AND OVERVIEW	6
Increasing the Speed and Safety of C Strings	6
Key Benefits	6
Using the Routines	6
DECLARING VARIABLES	7
Purpose	7
PL/I Style Variables	7
Notes	7
Examples	7
THE CAT, CATCHAR AND CATLIT FUNCTIONS	8
Purpose	8
Declaring Variables	8
Notes	8
Examples	8
THE CLEAR FUNCTION	9
Purpose	9
Declaring Variables	9
Notes	9
Examples	9
THE CMP, CMPCHAR AND CMPLIT INSTRUCTIONS	10
Purpose	10
Declaring Variables	10
Notes	10
Examples	10
THE CPY, CPYCHAR AND CPYLIT INSTRUCTIONS	11
THE CPY, CPYCHAR AND CPYLIT INSTRUCTIONS	11
Purpose	11
Declaring Variables	11
Notes	11
Examples	11
THE SET INSTRUCTIONS	12
Purpose	12

Declaring Variables	12
Notes	12
Examples	12
 VARIABLE BLOCK (VB) FILE INTEGRATION & RECORD I/O	 13
Record Input and Output.	13
Direct Mapping to VB Records	13
Opening Files VB Files	14
Notes	14
Reading Data and Writing Fast Safe Variables.	14
Notes.	14
Example	15
Notes.	15
Example 2	16

Introduction and Overview

Increasing the Speed and Safety of C Strings

Standard C string handling relies on null-terminated character arrays (`\0`), which presents two major drawbacks:

1. **Performance Overhead:** Operations like copying (`strcpy`) or comparing (`strcmp`) require scanning memory byte-by-byte to locate the trailing null terminator, missing out on hardware-accelerated block memory moves.
2. **Safety Risks:** Standard functions lack implicit boundary enforcement, making buffer overflows and memory corruption frequent bugs in production code.

The **Fast and Safe C String** library replaces null-terminated scanning with length-prefixed varying strings (`charvar`). By maintaining the explicit current length at the head of the string buffer, string operations bypass trailing null searches and execute via fast block memory transfers (`memcpy`, `memcmp`, or platform-native assembly instructions).

Key Benefits

- **Speed:** Leverages direct length counters to execute string operations using optimal block transfers.
- **Safety:** Automatic boundary checking prevents buffer overruns by truncating copies that exceed target capacity.
- **Modern Compiler Support:** Sometimes uses `static inline` functions and compiler optimizations across ANSI C platforms.

Using the Routines

To use the routines, you use special macros to define your variables, copy and compare strings, and read and write files.

Declaring Variables

Purpose

The Declare or DCL statement defines and initializes variables. It follows this form:

```
dcl (name,length) ;
```

where

name is the name of the variable being declared.

PL/I Style Variables

When you define a variable, you define to the **CPY**, **CMP** and other macros that the variable is a special style.

Notes

1. Strings are handled quite differently from "C" strings. The macros define the maximum lengths of the strings, and contain a special length integer that contains the current length of the string, resulting in improved performance when copying, comparing etc.. one string to another.
2. For strings, the name you specify is defined as a *const* pointer to another variable containing the actual string. This does two things: it ensures that you cannot accidentally overwrite the variable, and also allows the macro to define the string in a special format depending on the type of string (fixed length, or long or short varying strings).
3. You can use the **SET** instruction to initialize the variable.

Examples

1. **dcl (name,20);**

The declaration above describes a variable called *name*. *Name* is a varying length string with a maximum length of 20 characters.

2. **dcl (addr,80);**

SET (addr,"12 Main St");

The declaration above describes a variable called *addr*. *Addr* is a varying length string with a maximum length of 80 characters. Because this is a varying length string, the *current* length of the string is held in the first word.

The CAT, CATCHAR and CATLIT Functions

Purpose

The **CAT** function concatenates one variable with another. It follows this form:

```
cat (dest,src)
```

```
catchar (dest,'char');
```

```
catlit (dest,"literal")
```

where *dest* and *src* are variables defined with the **DCL**, **VAR** or **CSTR** macros.

Declaring Variables

Before using the generic **CAT** function, you must use the appropriate declaration macro to define the variable.

Notes

1. The **CAT** and **CATLIT** macro appends the source variable to the destination variable.
2. If the string is already at maximum length, the concatenation will have no effect.

Examples

1. **dcl (name,20);**

```
catlit(name,"");
```

The declaration above describes a variable called *name*. *Name* is a varying length string with a maximum length of 20 characters. The **catlit** adds a "," to the string *name*.

The CLEAR Function

Purpose

The **CLEAR** function sets a variable to 0, a null string. It follows this form:

clear (*variable*)

where *variable* is a variable defined with the **DCL** macro.

Declaring Variables

Before using the generic **CLEAR** function, you must use the appropriate declaration macro to define the variable.

Notes

1. The **CLEAR** macro sets binary numbers to 0, varying length strings to a null ("") string.
2. You may use the **CLEAR** macro to initialize the strings correctly, or copy the string to itself.

Examples

1. **dcl (name,20);**
clear(name);

The declaration above describes a variable called *name*. *Name* is a varying length string with a maximum length of 20 characters. The **clear** sets the string to *null*, and sets the current length to 0.

The CMP, CMPCHAR and CMPLIT Instructions

Purpose

The **CMP**, **CMPCHAR** and **CMPLIT** function compares one variable to another. It follows this form:

```
cmp (lhs_variable,rhs_variable)
```

```
cmpchar (dest,'char');
```

```
cmplit (lhs_variable,"literal-string")
```

where *lhs_variable* is a variable defined with the **DCL**, **VAR** or **CSTR** macros.

Declaring Variables

Before using the generic **CMP** instructions, you must use the appropriate declaration macro to define the variable.

Although the definitions of the strings look similar, the **CMP** and **CMPLIT** macros perform slightly different actions depending on the type of variable being used.

When a string is defined as a "C" string, the **STRLEN** function is used to find the length of the string *every time*. This means that there is no advantage in using this type of string - although it does allow you to import "C" variables easily.

Notes

1. The **CMP** macro compares the source variable to the destination variable.
2. Strings are handled quite differently from "C" strings. The macros define the maximum lengths of the strings, and contain a special length integer that contains the current length of the string, resulting in improved performance when copying, comparing etc. one string to another.

Examples

1. **dcl (lang,20);**

```
if (cmplit(lang,"C")==0) ...
```

The declaration above describes a variable called *lang*. *Lang* is a fixed length string with a maximum length of 20 characters. The **cmplit** compares the value in the string *lang* to a literal "C", and if it is equal, executes the next instruction.

2. **dcl (addr,80);**

The declaration above describes a variable called *addr*. *Addr* is a varying length string with a maximum length of 80 characters. Because this is a varying length string, the *current* length of the string is held in the first word.

The CPY, CPYCHAR and CPYLIT Instructions

Purpose

The **CPY** instruction copies one variable to another. It follows this form:

```
cpy      (destination,source) ;
```

```
cpychar (dest,'char');
```

```
cpylit   (destination,"literal") ;
```

where *destination* and *source* are variables defined with the **DCL**, **VAR** or **CSTR** macros.

Declaring Variables

Before using the generic **CPY** instructions, you must use the appropriate declaration macro to define the variable.

Although the definitions of the strings look similar, the **CPY** and **CMP** macros perform slightly different actions depending on the type of variable being used.

When a string is defined as a "C" string, the **STRLEN** function is used to find the length of the string *every time*. This means that there is no advantage in using this type of string - although it does allow you to import "C" variables easily.

Notes

1. The **CPY** macro copies the source variable to the destination variable.
2. Strings are handled quite differently from "C" strings. The macros define the maximum lengths of the strings, and contain a special length integer that contains the current length of the string, resulting in improved performance when copying, comparing etc. one string to another.

Examples

1. **dcl (name,20);**

The declaration above describes a variable called *name*. *Name* is a fixed length string with a maximum length of 20 characters.

2. **dcl (addr,80);**

The declaration above describes a variable called *addr*. *Addr* is a varying length string with a maximum length of 80 characters. Because this is a varying length string, the *current* length of the string is held in the first word.

The SET Instructions

Purpose

The **SET** instruction initialises or copies a string to a variable. It follows this form:

```
set      (variable, "literal string") ;  
where variable is defined with the DCL macros.
```

Declaring Variables

Before using the **SET** instruction, you must use the **DCL** declaration macro to define the variable.

Notes

1. The **SET** macro copies the literal string to the destination variable.
2. Strings are handled quite differently from "C" strings. The macros define the maximum lengths of the strings, and contain a special length integer that contains the current length of the string, resulting in improved performance when copying, comparing etc. one string to another.

Examples

1. **dcl (name,20);**
set (name,"Mary Jane");

The declaration above describes a variable called *name*. *Name* is a variable length string with a maximum length of 20 characters. The **SET** instruction copies the literal to **name** and set the length to 9.

Variable Block (VB) File Integration & Record I/O

Record Input and Output. The `Fast String` structure maps directly to variable-length file records, such as mainframe Variable Blocked (VB) datasets and length-prefixed record files on modern operating systems.

GET_REC and **PUT_REC** will read and write DCLED variables.

VB_Get, **VB_Put** and **VB_GetLocate** will read and write data from regular C variables and structures without using the fast string code.

These are very high speed file input/output functions that operate many times faster than standard **FGETS** and similar functions.

Direct Mapping to VB Records

In Variable Block (VB) processing, each record is prefixed with a length descriptor (e.g., a 2-byte or 4-byte Record Descriptor Word / RDW). Because `FSS` strings manage length explicitly at the head of the buffer rather than relying on standard null termination (`\0`):

1. **No Data Truncation on Binary Zeros:** Records containing embedded binary zeros (e.g., packed decimal or raw binary fields) can be read directly into a `charvar` buffer without being prematurely terminated.
2. **Zero-Copy I/O:** Record length information read from a VB file header directly initializes the length field of a `charvar` variable. Conversely, writing a `charvar` variable to a VB stream uses the string's length header to set the block output record size automatically.
3. **Efficiency:** Eliminates the need to pad record buffers with spaces or scan buffers with `strlen` prior to writing output blocks.

The files are in IBM Variable Blocked format. They can be transferred to a mainframe, and read as **RECFM=FB** format files.

Before the Variable Blocked files can be read or written, they must be opened. Optionally, the files can be translated to or from ASCII and EBCDIC.

Opening Files VB Files

```
File handle VB_OpenRead(path, translate);  
File handle VB_OpenWrite(path, block_size, translate);  
Or generic Open:  
File handle VB_Open(path, mode, block_size);
```

where **File Handle** is the name used to access the file in the PUT and GET function

and **path** is the name of the file name or DDNAME

and **block size** is the block size of the file (Note, Read files automatically are allocated a buffer of 32K)

and **translate** are optional conversion tables to convert from Ascii to Ebedic. Two tables are provide – 037 and 1047.

Notes

Reading Data and Writing Fast Safe Variables. GETREC and PUTREC will read and write FSS variables

```
GETREC (File handle,variable);  
PUTREC(File handle,variable);
```

where **File Handle** is the name used to open and close files.

and variable is a variable created with the DCL function.

These functions return a 0 for successful operation, otherwise a negative number.

Notes.

GET_REC(handle, name)

1. Read one **VB** record into **DCL**'d buffer 'name'.
2. Automatically updates **dv_##name.cur_len** from the record length.
3. Returns the number of bytes read, or <= 0 at end of file / error.
4. Check the return value directly if you need status:
if (GET_REC(h, buf) <= 0) { ... EOF or error ... }

Example

```
int main() {
    vb_handle_t *in = VB_OpenRead("input.vbf","");
    vb_handle_t *out = VB_OpenWrite("out.vbf", 32768, "");
    if (!in || !out) return 1;

    DCL(rec_buf, 512);
    DCL(work_area, 600);
    uint32_t bytes_read;
    uint32_t records_processed = 0;

    while (GETREC(in, rec_buf) > 0) {
        // Fast Concatenation: O(1) complexity (No scanning)
        SET(work_area, "PROC:");
        CAT(work_area, rec_buf);

        // Direct Write: We already know the length
        Putrec(out, work_area);
        records_processed++;
    }

    VB_Close(in); VB_Close(out);
```

GET_REC (File handle,variable);

PUT_REC(File handle,variable);

where *File Handle* is the name used to open and close files.

and variable is a variable created with the DCL function.

These functions return a 0 for successful operation, otherwise a negative number.

Notes.

Example 2

```
Int main() {
    vb_handle_t *in = VB_OpenRead("input.vbf", "");
    vb_handle_t *out = VB_OpenWrite("out.vbf", 32768, "");
    if (!in || !out) return 1;

    DCL(rec_buf, 512);
    DCL(work_area, 600);
    uint32_t bytes_read;
    uint32_t records_processed = 0;

    while (VB_Get(in, rec_buf, 512, &bytes_read) > 0) {
        dv_rec_buf.cur_len = bytes_read; // Sync Dope Vector

        // Fast Concatenation: O(1) complexity (No scanning)
        SET(work_area, "PROC:");
        CAT(work_area, rec_buf);

        // Direct Write: We already know the length
        // (dv_work_area.cur_len)

        VB_Put(out, work_area, dv_work_area.cur_len);
        records_processed++;
    }

    VB_Close(in); VB_Close(out);
}
```


Fast and Safe C Strings.

Readers Comment Form

This manual is part of the Oscar library that serves as a reference source for Managers, Systems Analysts, Programmers and Operators. This form may be used to communicate your views about this publication.

Clarke Computer Software may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation whatever. You may, of course, continue to use any information you supply.

Possible topics for comments are:

Clarity Accuracy Completeness Organization Legibility

If you wish a reply, give your name and address or Email address

Number of latest Newsletter associated with this publication:

Please send your comments to:

Email admin@Oscar-Jol.com